



14b-protection-security

Koruma ve Erişim Kontrolü

Koruma Nedir?

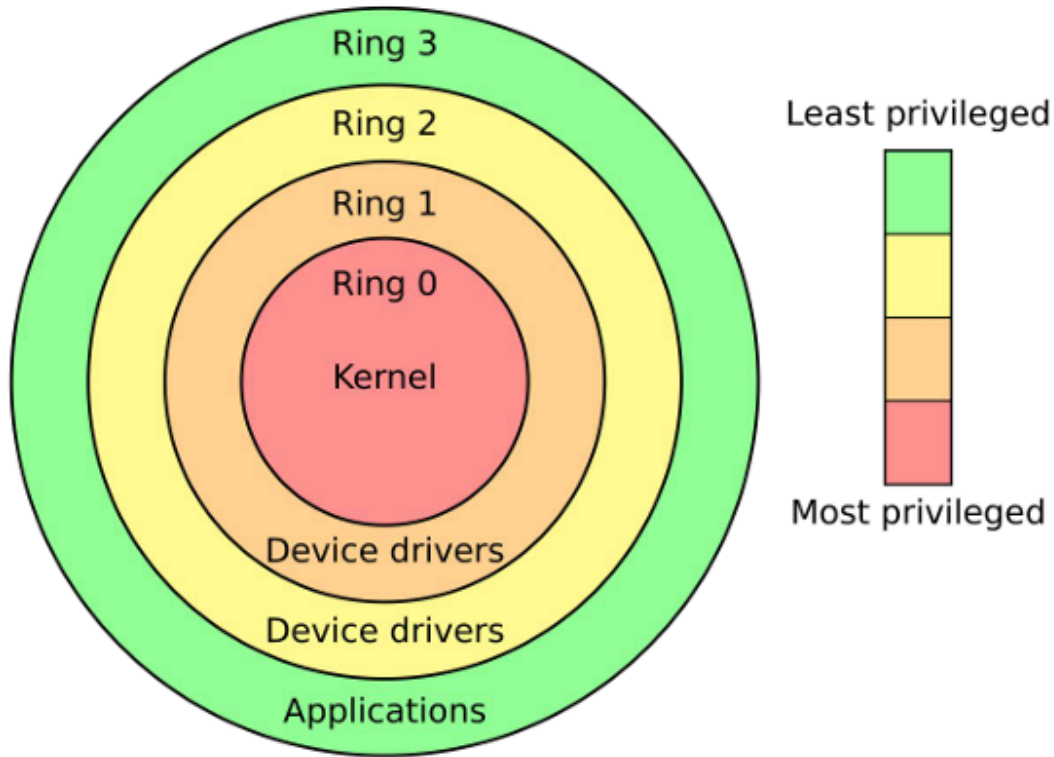
Koruma sistemi, bir sistemin hangi koşullarda güvenli olduğunu tanımlar. Bilgisayar, donanım veya yazılım **nesnelerinden** oluşur. Her nesneye sadece yetkili süreçlerin doğru şekilde erişmesi korumanın hedefidir.

- **En az ayrıcalık prensibi:** Programlara, kullanıcılara ve sistemlere, görevlerini yapabilmeleri için gereken **en az** yetki verilmelidir. Bu, hata veya kötüye kullanımda hasarı sınırlar.
- **Denetim izi (audit trail):** Koruma ile ilgili tüm eylemler kaydedilmelidir.

Koruma Halkaları (Protection Rings)

Multics'ten gelen hiyerarşik koruma modeli.

- Donanım desteği gerekir.
- x86'da 4 halka (0 en yetkili, 3 en az yetkili) vardır, ancak çoğu işletim sistemi sadece halka 0 (çekirdek) ve halka 3 (kullanıcı) kullanır.
- ARM TrustZone, kriptolojik işlemleri için ek bir güvenli halka ekler (Android Trusty TEE).



MAC & DAC

DAC (Discretionary Access Control)

Nesnenin sahibi, erişimi kendi takdirine göre belirleyebilir. Örneğin Unix dosya izinleri, Windows ACL.

Zayıflığı: Kullanıcı yetkilerini kötüye kullanabilir (örneğin bir dosyayı izinsiz paylaşabilir).

MAC (Mandatory Access Control)

Sistem, bir politikaya göre erişimi zorunlu olarak kısıtlar. Nesne ve öznelere **etiketler** atanır. **Root bile MAC'i geçemez.** Örnekler: SELinux, TrustedBSD, Windows Vista MAC. MAC, gizli bilgilerin sızdırılmasını önlemek için kullanılır (askeri sistemler).

Diğer Koruma Yöntemleri

- **Sistem bütünlüğü koruması (SIP):** macOS'ta root dahil sistem dosyalarına erişimi kısıtlar.
- **Sistem çağrısı filtreleme:** Linux SECCOMP-BPF ile sistem çağrılarını kısıtlama.
- **Sandboxing:** Süreci kısıtlı bir ortamda çalıştırma (Java VM, .NET, macOS seatbelt).

İki Seviyeli Yaklaşımlar

İşletim sistemlerinde ince taneli erişim kontrolü uygulamak sistemi çok karmaşık hale getirebilir.

Bu nedenle OS, **ayrıcalıklı programlar** (setuid, setgid, daemon'lar) aracılığıyla ince taneli kontrolü dışarıya devreder. Bu programlar, normal kullanıcıların ihtiyaç duyduğu özel işlemleri (şifre değiştirme gibi) güvenli şekilde yapmalarını sağlar.

Deamon'lar ve Setuid Programları

- **Daemon:** Arka planda çalışan ve genellikle root veya başka ayrıcalıklı kullanıcı yetkileriyle çalışan programdır.
- **Setuid programı:** Dosya üzerinde `setuid` biti işaretlenmiş çalıştırılabilir dosyadır. `ls -l` çıktısında `s` ile gösterilir (örneğin `rws--x--x`). Dosyanın sahibi (genelde root) olmalıdır; setgid için de dosya grubunda olmak gerekir.

Deamon

Bir daemon, bilgisayar açıldığından itibaren **arka planda çalışan**, ekranı veya kullanıcı etkileşimi olmayan, sürekli hizmet veren bir programdır.

Görevleri:

- İstekleri dinlemek (ağ bağlantıları, zamanlanmış görevler)
- Sistem hizmetlerini yürütmek (yazdırma kuyruğu, günlük kaydı, zaman senkronizasyonu)

Setuid Mekanizması

Setuid, normal kullanıcıların geçici olarak yükseltilmiş yetkilerle program çalıştırmasına izin verir.

- **Gerçek UID (RUID):** Programı başlatan kullanıcının kimliği. Değişmez.
- **Etkin UID (EUID):** Erişim kontrollerinde kullanılan kimlik.
- **Normal program:** Çalıştıran kullanıcının yetkileriyle çalışır.
- **Setuid program:** Çalıştıran kullanıcının değil, **dosya sahibinin** yetkileriyle çalışır.

Setuid Programlarının Saldırı Yüzeyleri

1. Kullanıcı Girdileri Yoluyla Saldırıları

- **Buffer Overflow (arabellek taşması):** Program, kullanıcının girdiği veriyi yeterli boyut kontrolü yapmadan bir arabelleğe kopyalarsa, saldırgan fazla veri göndererek dönüş adresini değiştirebilir ve kötü amaçlı kod çalıştırabilir.
- **Format String Vulnerability:** Program, kullanıcı girdisini `printf` gibi fonksiyonlara doğrudan format parametresi olarak verirse (örneğin `printf(user_input)` yerine `printf("%s", user_input)` yapılmazsa), saldırgan `%x`, `%n` gibi format belirteçleri göndererek belleği okuyabilir veya yazabilir.

2. Sistem Girdileri Yoluyla Saldırıları

- **Race Condition (Yarış Durumu):** Özellikle **TOCTTOU (Time of Check to Time of Use)** hatası. Program bir dosyayı kontrol eder (örneğin erişim izni var mı diye bakar) ile o dosyayı kullanması arasında geçen sürede, saldırgan dosyayı değiştirir (örneğin sembolik link ile başka bir dosyaya yönlendirir).
- **Symbolic Link saldırıları:** Program dünyada yazılabilir bir klasörde (örneğin `/tmp`) işlem yapıyorsa, saldırgan bu klasörde sembolik linkler oluşturarak programı kandırabilir.

İzolasyon

Bell-La Padula Modeli

MAC'in en bilinen örneği. **Bilgi akışını kontrol eder.** Amacı, gizli bilgilerin daha düşük seviyeli kullanıcılara sızmasını engellemek. İki temel kural:

1. **No-read-up (Yukarı okuma yok):** Bir kullanıcı, kendi seviyesinden daha yüksek gizlilik seviyesindeki bir nesneyi okuyamaz.
2. **No-write-down (Aşağı yazma yok):** Bir kullanıcı, kendi seviyesinden daha düşük gizlilik seviyesindeki bir nesneye yazamaz (çünkü bu, yüksek seviyeli bilginin düşük seviyeye sızmasına yol açar).

Bu model, askeri sistemlerde kullanılır.

Low-Water Mark (LOMAC) ve Mandatory Integrity Control

- **LOMAC (Low-Water Mark Access Control):** Bell-La Padula'nın bütünlük odaklı bir varyasyonudur. Bir süreç, düşük bütünlüklü bir nesneyi okuduktan sonra, kendi bütünlük seviyesi düşer. Bu sayede yüksek bütünlüklü verinin kirlenmesi önlenir. Linux 2.2'de vardı, şimdi FreeBSD'de mevcut.
- **Windows Mandatory Integrity Control (MIC):** Benzer bir mekanizma. Düşük bütünlüklü bir işlem, yüksek bütünlüklü nesnelere yazamaz.

SELinux (Security-Enhanced Linux)

SELinux, MAC'in Linux'a uygulamasıdır. Üç temel kavramı vardır:

- **Kullanıcılar (Users):** SELinux kullanıcıları (Unix kullanıcılarından farklıdır), login sırasında kurallara göre atanır.
- **Roller (Roles):** Bir kullanıcı, `newrole` komutuyla farklı rollere bürünebilir. Roller, SELinux (Unix değil) kullanıcıları tarafından kısıtlanır.
- **Tipler (Types):** Her role, yalnızca belirli türlere izin verir. Türler, hiyerarşik olmayan güvenlik politikaları sağlar. Örneğin `httpd_t` türüne sahip bir süreç, sadece `httpd_sys_content_t` türündeki dosyaları okuyabilir.

Güvenlik ve Bi şey

A. Speculative Execution (Spekülatif Çalıştırma):

İşlemcinin (CPU) performansı artırmak ve bellek okuma gecikmelerini (latency) maskelemek amacıyla, programın hangi yöne dallanacağını önceden tahmin ederek sıradaki komutları erkenden (kesinleşmeden önce) çalıştırması mimarisidir. Tahmin yanlış çıkarsa yapılan işlemler geri alınır ve doğru yoldan devam edilir. Bu bir koruma yöntemi değil, Spectre gibi donanım seviyesi sızıntı saldırılarına (Microarchitectural attacks) zemin hazırlayabilen bir işlemci özelliğidir.

B. Canaries (Kanaryalar):

Derleyiciler (compiler) tarafından programa eklenen bir "Stack-Guard" (Yığıt Koruması) mekanizmasıdır. Bellekteki yığıt (stack) alanında, kritik geri dönüş adreslerinin (return address) hemen önüne yerleştirilen küçük, rastgele kontrol değerleridir. Bir arabellek taşması (buffer overflow) durumunda, dışarıdan gelen veri taşıp asıl adresi bozmadan hemen önce bu "kanarya" değerini ezip bozacağı için, sistem saldırıyı anında tespit eder ve programı güvenli bir şekilde sonlandırır.

C. Address Space Layout Randomization (ASLR - Adres Uzayının Rastlaştırılması):

İşletim sistemi düzeyinde uygulanan bir güvenlik önlemidir. Bir program belleğe her yüklendiğinde, yığıtın (stack) ve diğer kritik veri alanlarının bellek üzerindeki başlangıç adreslerini sürekli olarak rastgele değiştirir. Bu sayede saldırganların zararlı kodlarını yönlendirecekleri bellek adreslerini ezberden bilmeleri veya önceden tahmin etmeleri engellenir.

D. Non-executable Stack (Çalıştırılmayan Yığıt):

Donanım (Hardware) düzeyinde desteklenen bir koruma yaklaşımıdır. Bellekteki yığıt (stack) bölgesinin yalnızca "veri depolama" amacıyla işaretlenmesini prensibine dayanır. Saldırgan bir zafiyet bulup yığıt alanına kendi zararlı kodunu sızdırmayı başarsa bile, bu önlem sayesinde sistem o bölgedeki verilerin komut olarak "çalıştırılmasına" (execute edilmesine) kesinlikle izin vermez.